

Document de synthèse - Projet Sokoban

Erhan BAYRAKCEKEN | mai 2026

Palier visé

Palier C – L'application gère les 5 niveaux tutoriels (palier A), les 10 niveaux standards de tailles variables (palier B), ainsi que le chargement de niveaux externes au format .xsb depuis un dossier sélectionné par l'utilisateur (palier C).

Options implémentées

Difficulté	Option
*	Compteur de poussées (en plus des mouvements)
*	Nom du niveau affiche (titre de la fenêtre)
**	Annuler (Undo) via bouton, menu et Ctrl+Z
**	Navigation entre niveaux (boutons Précédent / Suivant)
**	Sélection du niveau via menu déroulant dynamique
**	Palier B -- 10 niveaux varies, grille construite dynamiquement (Niveaux)
***	Progression automatique après victoire (activable / désactivable)
***	Palier C -- chargement de dossier .xsb via DirectoryChooser
****	Sauvegarde des scores dans un fichier local (Scores.sav)

Catégorie JavaFX / IHM

Difficulté	Option
*	Personnalisation CSS (thème sombre, couleurs, style des boutons). Le CSS a été réalisé avec l'aide de l'IA.
*	Menu "A propos" avec nom et date de mise à jour
**	Raccourcis clavier (Z/Q/S/D, flèches, R, N, P, X, C, U, A, T, Ctrl+Z)
**	MenuBar structurée (Fichier / Jeu / Niveaux / ? (Aide et à Propos)
***	Affichage avec sprites (ImageView) pour chaque élément du jeu
***	Compteurs réactifs avec IntegerProperty et binding
***	Indicateur de progression "Caisses placées : X/Y"
****	Sons : Poussée, victoire, défaite, caisse sur cible, musique de fond en boucle (fermable avec A)

Présentation rapide de l'interface

L'interface est construite avec FXML (Scene Builder) et organisée autour d'un BorderPane principal. En haut, une barre de menus (MenuBar) donne accès aux différents modes de jeu (Tutoriel, Niveaux Normaux, Charger un dossier .xsb), aux actions de jeu (annuler, recommencer, niveau suivant/précédent) ainsi qu'à la sélection directe d'un niveau via un menu déroulant génère dynamiquement. Juste en dessous, un bandeau de statistiques affiche en temps réel le numéro de niveau, le nombre de caisses placées, les mouvements et les poussées via des compteurs réactifs (IntegerProperty + binding).

Au centre, la grille de jeu est générée dynamiquement via un GridPane dont les dimensions s'adaptent automatiquement au niveau chargé. Chaque case est représentée par un sprite (ImageView de 40x40 pixels). Le joueur dispose de sprites directionnels différents selon qu'il marche ou qu'il pousse une caisse (12 images gérées via deux HashMap), y compris lorsqu'il se trouve sur une cible. En bas, trois boutons (Précédent, Recommencer, Suivant) se désactivent automatiquement selon la position dans la banque de niveaux.

La gestion audio repose sur une classe enum EffetSFX qui centralise tous les effets sonores (déplacement, poussée, caisse sur cible, victoire, défaite). Les sons sont chargés une seule fois au démarrage via AudioClip pour éviter toute latence (même si cela n'a pas vraiment marché finalement). Une musique de fond tourne en boucle et se relance à chaque changement de niveau ou redémarrage. Des alertes gèrent les fins de niveau (victoire, Game over, dernier niveau termine), la confirmation de fermeture et les erreurs de chargement. Toutes les SokobanException remontées par le moteur sont interceptées et présentées à l'utilisateur sans provoquer de crash. A chaque fermeture, les scores de la session sont écrits dans un fichier Scores.sav en UTF-8, avec pour chaque niveau le nom, le nombre de mouvements, de poussées et de caisses placées.